

Programação e Desenvolvimento de Software I

Arquivos

Prof. Héctor Azpúrua
(slides adaptados do Prof. Pedro Olmo)

Arquivos

Temporal vs permanente

- Sabemos que o armazenamento de **dados** em **variáveis** é **temporário**
 - Os dados se perdem quando o programa termina sua execução!
- E se precisamos reutilizar esses dados?
 - Podemos usar **arquivos**!
 - Conjunto de dados digitalizados que pode ser gravado em um dispositivo de armazenamento e tratado como ente único
- **Arquivos**, são usados para manter grandes quantidades de dados de **forma permanente**!
 - Persistência de informações
 - Volumes maiores de dados
 - Podem ser usados como entrada do nosso programa

Uso de arquivos

- Aplicações empresariais como, por exemplo, controle de estoque, processam muitos dados e não seriam possíveis sem o uso de arquivos
- Os **arquivos** são armazenados em dispositivos de **memória secundária**:
 - Fitas magnéticas, discos magnéticos, discos óticos e memórias de estado sólido
- Estes dispositivos podem reter **grandes volumes de dados por longos períodos de tempo**



Arquivos

Problema I

- Considere que um arquivo de dados contém os valores das dimensões (tam. max.: 100) e dos elementos de duas matrizes de números inteiros. Ex:

2 5

4 12 1 78 5

98 9 1994 0 52

79 12 458 2 29

47 19 784 12 8

- Implemente um programa que calcule e exiba a matriz resultante da soma de duas matrizes (tam. max.: 100) lidas de um arquivo

Arquivos

Problema I

Implemente um programa que calcule e exiba a matriz resultante da soma de duas matrizes (tam. max.: 100) lidas de um arquivo

■ **Algoritmo:**

1. Criar duas matrizes
2. Ler o arquivo
3. Armazenar os dados do arquivo nas matrizes
4. Fazer a soma das matrizes
5. Imprimir a matriz resultante

Arquivos

Problema I

Implemente um programa que calcule e exiba a matriz resultante da soma de duas matrizes (tam. max.: 100) lidas de um arquivo

■ Algoritmo:

1. Criar duas matrizes 😊
2. Ler o arquivo 😞
3. Armazenar os dados do arquivo nas matrizes 😞
4. Fazer a soma das matrizes 😊
5. Imprimir a matriz resultante 😊



Uso de arquivos

Vantagens

- Considere, por exemplo, que o problema recebesse duas matrizes de dimensões **10x10**
 - Se a entrada de dados fosse feita interativamente, o usuário teria que digitar **200 valores!!!**
- Estes valores deveriam ser digitados novamente a cada execução do programa
 - Portanto, é mais fácil armazenar estes valores em arquivo e, sempre que o programa for executado, efetuar a leitura dos valores partir do arquivo

Manipulação de arquivos

- A linguagem C não impõe estrutura alguma aos arquivos
 - Do **ponto de vista físico**, um arquivo é, simplesmente, uma **sequência de bytes**
- Mas, do **ponto de vista lógico**, costuma-se imaginar que os dados armazenados em **arquivos são organizados em registros**
- Considere, por exemplo, um **sistema de folha de pagamento**

Manipulação de arquivos

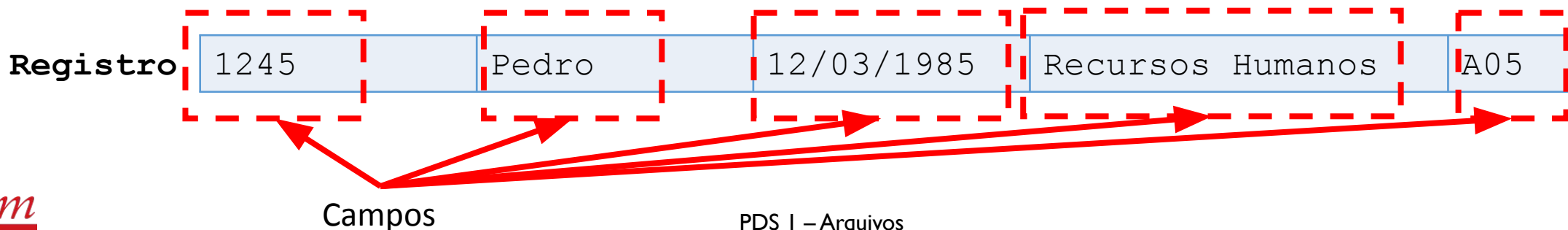
Sistema folha de pagamento

- Em um sistema como esse, cada **registro** corresponde a um **empregado** e pode conter, por exemplo, os seguintes dados:
 - Número de matrícula;
 - Nome do empregado;
 - Data de início na empresa;
 - Departamento onde trabalha;
 - Categoria funcional.
- Cada um dos dados que compõem um registro denomina-se **campo**

Manipulação de arquivos

- A figura abaixo ilustra a organização lógica de dados em um arquivo:

Arquivo	1245	Pedro	12/03/1985	Recursos Humanos	A05
	1367	Henrique	03/01/1994	Financeiro	A01
	1380	Filipe	31/05/2002	Planejamento	B18
	1432	Bruno	28/11/1999	Administração	B11



Manipulação de arquivos

Identificadores únicos de registro

- Normalmente, a recuperação de dados em um arquivo é feita **registro a registro**
- Para facilitar a recuperação, pelo menos um campo deve, univocamente, **identificar o registro**
 - Este campo especial, que identifica um registro específico, é conhecido como **chave de registro**
- No exemplo anterior, o número de matrícula pode ser a chave do registro

Registro	1245	Pedro	12/03/1985	Recursos Humanos	A05

Arquivos textuais

- Para utilizar um arquivo de dados, é preciso declarar um ponteiro para o tipo **FILE**
- Isso é feito da seguinte forma:

FILE ***arq**;

- Com isso, a variável **arq** passa a representar, dentro do programa, um arquivo de dados
- É importante observar que um arquivo de dados é reconhecido pelo sistema operacional como uma entidade externa ao programa

Arquivos textuais

- Antes que um arquivo possa ser utilizado, é preciso informar ao sistema operacional onde localizar o arquivo e como se pretende usar o arquivo...
 - Esta operação é conhecida como abertura do arquivo e é executada pela função `fopen`

```
arq = fopen("/meus_docs/matrizes.txt", "w");
```

- A função `fopen` tem dois parâmetros do tipo `string`:
 1. O primeiro parâmetro especifica o **nome/localização** do arquivo
 2. O segundo parâmetro especifica o que **deve ser feito** com os dados

Arquivos textuais

Usando `fopen`

- Localizando os arquivos:

```
arq = fopen("/meus_docs/matrizes.txt", "w");
```

- O nome do arquivo é `matrizes.txt`
- E se encontra dentro da pasta `/meus_docs/`

- E neste caso, **onde** se encontra o **arquivo**?

```
arq = fopen("matrizes.txt", "w");
```

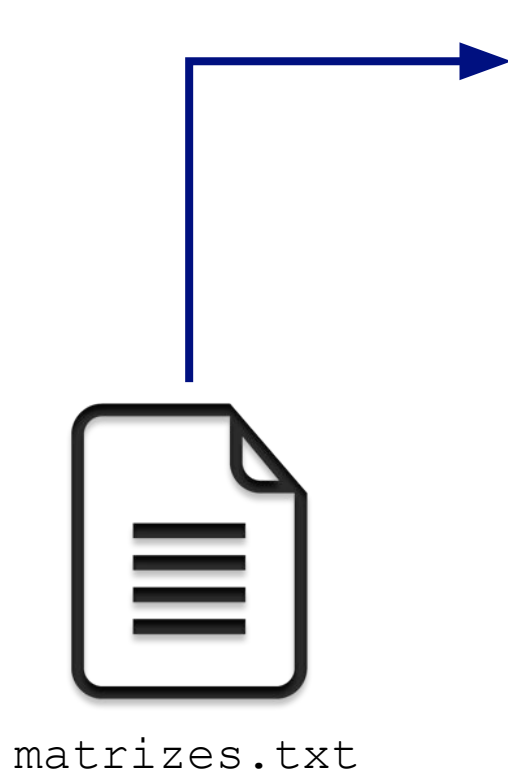
- Como **não foi fornecida sua localização**, este arquivo deve estar presente na mesma pasta que contém o arquivo executável gerado (ex: `a.exe`)
- Caso o arquivo não esteja na mesma pasta que o programa executável, é preciso informar a pasta que o contém!

Arquivos textuais

- A abertura de um arquivo, se bem sucedida, retorna em uma estrutura do tipo **FILE** várias informações (dados de ponteiros de arquivo)
 - Se, por alguma razão, o arquivo não puder ser aberto, a função `fopen` irá retornar a constante `NULL`
- Essa estrutura é um ponteiro para permitir que seus campos sejam alterados por outras funções (ex: `fscanf`)
- Alguns exemplos dessas informações são:
 - Um ponteiro para o buffer associado ao arquivo
 - O tamanho do buffer
 - A indicação se o buffer está cheio ou vazio
 - Um ponteiro para o dado atual no arquivo

Arquivos textuais

Abrindo um arquivo com `fopen`



```
FILE *arq;  
arq = fopen("matrizes.txt", "w");
```

Endereço	Variável	Conteúdo
0xFF12	arq.campo1	0xFFA4
0xFF13	arq.campo2	1024
0xFF14	arq.campo3	0
0xFF15	arq.campo4	0xFFBB
...	...	...
0xFF (n)	arq.campo (n)	48

`arq == 0xFF12`

Arquivos textuais

- O segundo parâmetro da função `fopen` especifica o que **deve ser feito** com os dados
- As possibilidades são:
 - Ler os dados existentes “r”
 - Gravar dados apagando (caso existam) os dados existentes “w”
 - Anexar novos dados aos já existentes “a”
- A instrução abaixo especifica o modo “r”, ou seja, “ler” (“read”) os dados existentes:

`arq = fopen("matrizes.txt", "r");`



Arquivos textuais

- A tabela a seguir mostra os possíveis modos de abertura de um arquivo:

Modo	Significado
"r"	Abre um arquivo existente para leitura de dados; se o arquivo não existir, irá ocorrer um erro
"w"	Abre um novo arquivo para gravação de dados; se o arquivo já existir, a gravação irá sobrescrever os dados existentes
"a"	Abre um arquivo para operações de anexação de dados; se o arquivo não existir, será criado um novo arquivo

Arquivos

Problema I

- Considere que um arquivo de dados contém os valores das dimensões (tam. max.: 100) e dos elementos de duas matrizes de números inteiros. Ex:

2 5

4 12 1 78 5

98 9 1994 0 52

79 12 458 2 29

47 19 784 12 8

- Implemente um programa que calcule e exiba a matriz resultante da soma de duas matrizes (tam. max.: 100) lidas de um arquivo

Arquivos textuais

- Imagine que o arquivo `matrizes.txt` contenha os dados dispostos como a seguir:

2 5 ← Dimensões das matrizes

4 12 1 78 5 ← Matriz A

98 9 1994 0 52

79 12 458 2 29 ← Matriz B

47 19 784 12 8

- Para resolver o Problema I, estes dados são lidos pela função `fscanf`:

```
int linhas, colunas;  
fscanf(arq, "%d %d", &linhas, &colunas);  
printf("Linhas: %d, Colunas: %d\n", linhas, colunas);
```

Usando `fscanf`

```
int fscanf(FILE *stream, const char *format, ...);
```

- **FILE *stream**
 - Ponteiro para o objeto FILE que identifica o fluxo de entrada do qual os dados devem ser lidos
- **const char *format**
 - String de formato (similar ao printf contend o formato dos dados a serem lidos)
- **...**
 - Número variável de argumentos adicionais que apontam para os locais onde os valores analisados serão armazenados (ponteiros de variaveis)
- A função `fscanf` **retorna** um valor inteiro que representa o número de itens de entrada correspondidos e atribuídos com sucesso
 - Se a entrada terminar antes da primeira falha de correspondência ou se ocorrer um erro, EOF (End Of File) será retornado

Arquivos textuais

- A leitura das duas matrizes é feita, linha por linha, pelas instruções:

```
int M1[linhas][colunas];  
int M2[linhas][colunas];  
  
for (int i = 0; i < linhas; i++)  
    for (int j = 0; j < colunas; j++)  
        fscanf(arq, "%d", &M1[i][j]);  
  
for (int i = 0; i < linhas; i++)  
    for (int j = 0; j < colunas; j++)  
        fscanf(arq, "%d", &M2[i][j]);
```

- **Atenção!**

- Observe que a leitura deve ser feita de acordo com a disposição dos dados no arquivo

Problema I

Solução



matrizes.txt

2 5
4 12 1 78 5
98 9 1994 0 52
79 12 458 2 29
47 19 784 12 8

linhas=?

colunas=?

M1[linhas][colunas]=?

M2[linhas][colunas]=?

```
int main() {  
    FILE *arq;  
    arq = fopen("matrizes.txt", "r");  
  
    int linhas, colunas;  
    fscanf(arq, "%d %d", &linhas, &colunas);  
  
    int M1[linhas][colunas];  
    int M2[linhas][colunas];  
  
    for (int i = 0; i < linhas; i++)  
        for (int j = 0; j < colunas; j++)  
            fscanf(arq, "%d", &M1[i][j]);  
  
    for (int i = 0; i < linhas; i++)  
        for (int j = 0; j < colunas; j++)  
            fscanf(arq, "%d", &M2[i][j]);  
  
    for (int i = 0; i < linhas; i++)  
        for (int j = 0; j < colunas; j++)  
            M1[i][j] += M2[i][j];  
  
    for (int i = 0; i < linhas; i++) {  
        for (int j = 0; j < colunas; j++)  
            printf("%d ", M1[i][j]);  
        printf("\n");  
    }  
  
    fclose(arq);  
    return 0;  
}
```

Problema I

Solução



matrizes.txt

```
2 5
4 12 1 78 5
98 9 1994 0 52
79 12 458 2 29
47 19 784 12 8
```

linhas=2

colunas=5

M1[linhas][colunas]=

M2[linhas][colunas]=


```
int main() {
    FILE *arq;
    arq = fopen("matrizes.txt", "r");

    int linhas, colunas;
    fscanf(arq, "%d %d", &linhas, &colunas);

    int M1[linhas][colunas];
    int M2[linhas][colunas];

    for (int i = 0; i < linhas; i++)
        for (int j = 0; j < colunas; j++)
            fscanf(arq, "%d", &M1[i][j]);

    for (int i = 0; i < linhas; i++)
        for (int j = 0; j < colunas; j++)
            fscanf(arq, "%d", &M2[i][j]);

    for (int i = 0; i < linhas; i++)
        for (int j = 0; j < colunas; j++)
            M1[i][j] += M2[i][j];

    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < colunas; j++)
            printf("%d ", M1[i][j]);
        printf("\n");
    }

    fclose(arq);
    return 0;
}
```

Problema I

Solução



matrizes.txt

```
  2 5
4 12 1 78 5
98 9 1994 0 52
79 12 458 2 29
47 19 784 12 8
```

linhas=2

colunas=5

M1[linhas][colunas]=

M2[linhas][colunas]=


```
int main() {
    FILE *arq;
    arq = fopen("matrizes.txt", "r");

    int linhas, colunas;
    fscanf(arq, "%d %d", &linhas, &colunas);

    int M1[linhas][colunas];
    int M2[linhas][colunas];

    for (int i = 0; i < linhas; i++)
        for (int j = 0; j < colunas; j++)
            fscanf(arq, "%d", &M1[i][j]);

    for (int i = 0; i < linhas; i++)
        for (int j = 0; j < colunas; j++)
            fscanf(arq, "%d", &M2[i][j]);

    for (int i = 0; i < linhas; i++)
        for (int j = 0; j < colunas; j++)
            M1[i][j] += M2[i][j];

    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < colunas; j++)
            printf("%d ", M1[i][j]);
        printf("\n");
    }

    fclose(arq);
    return 0;
}
```

Problema I

Solução



matrizes.txt

```
2 5
4 12 1 78 5
98 9 1994 0 52
79 12 458 2 29
47 19 784 12 8
```

linhas=2

colunas=5

M1[linhas][colunas]=

83	24	459	80	34
145	28	2778	12	60

M2[linhas][colunas]=

79	12	458	2	29
47	19	784	12	8

```
int main() {
    FILE *arq;
    arq = fopen("matrizes.txt", "r");

    int linhas, colunas;
    fscanf(arq, "%d %d", &linhas, &colunas);

    int M1[linhas][colunas];
    int M2[linhas][colunas];

    for (int i = 0; i < linhas; i++)
        for (int j = 0; j < colunas; j++)
            fscanf(arq, "%d", &M1[i][j]);

    for (int i = 0; i < linhas; i++)
        for (int j = 0; j < colunas; j++)
            fscanf(arq, "%d", &M2[i][j]);

    for (int i = 0; i < linhas; i++)
        for (int j = 0; j < colunas; j++)
            M1[i][j] += M2[i][j];

    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < colunas; j++)
            printf("%d ", M1[i][j]);
        printf("\n");
    }

    fclose(arq);
    return 0;
}
```

Arquivos textuais

Problema I

- E se os dados de `matrizes.txt` estivessem nesse formato?

2 5

4 12 1 78 5 79 12 458 2 29

98 9 1994 0 52 47 19 784 12 8

M1

M2

- Como seria a leitura?

```
for (int i = 0; i < linhas; i++) {  
    for (int j = 0; j < colunas; j++)  
        fscanf(arq, "%d", &M1[i][j]);  
    for (int j = 0; j < colunas; j++)  
        fscanf(arq, "%d", &M2[i][j]);  
}
```

Usando fprintf

Escrevendo no arquivo

- Similar a sintaxe do fscanf, podemos usar fprintf para escrever num arquivo usando formatadores:

```
#include <stdio.h>

int main() {
    FILE *file = fopen("data.txt", "w");
    int ano = 2025;
    fprintf(file, "22/05/%d\n", ano);
    fclose(file);
}
```

Fechando arquivos

- Da mesma forma que um programa deve abrir um arquivo antes de usá-lo, **deve também fechar o arquivo** quando não precisar mais dele
- Na solução do Problema 1, o fechamento do arquivo é feito logo após a leitura das duas matrizes:

`fclose(arq);`

- O fechamento de um arquivo instrui o sistema operacional a:
 - Esvaziar os buffers associados ao arquivo
 - Liberar os recursos do sistema que o arquivo consumiu

Fechando arquivos

- Se o fechamento do arquivo for **bem sucedido**, a função `fclose` retorna o valor 0
- Se ocorrer um erro, `fclose` retorna a constante `EOF`, definida em `stdio.h` (igual a -1)
- Se não forem fechados pela função `fclose`, todos os arquivos abertos serão fechados quando o programa terminar sua execução

Perguntas?

- E-mail:
 - hector@dcc.ufmg.br
- Material da disciplina:
 - <https://pedroolmo.github.io/teaching/pdsI.html>
- Github:
 - <https://github.com/h3ct0r>



Héctor Azpúrua
h3ct0r